

Estruturas de Dados: Árvores

Professor: Elton Sarmanho¹

✉ eltonss@ufpa.br

¹  Faculdade de Sistemas de Informação – UFPA/CUTINS

7 de setembro de 2026

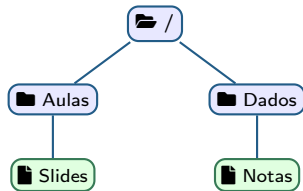
Roteiro da Aula

Objetivos de Aprendizagem

- ▶ O que é uma árvore e como ela difere de estruturas lineares?
- ▶ O que caracteriza uma árvore binária e uma árvore binária de busca?
- ▶ Quando uma ABB é eficiente? Quais são seus percursos principais?
- ▶ Por que a AVL precisa de rotações? Onde árvores de decisão se aplicam?

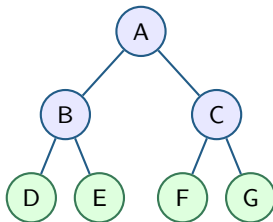
Por que usar árvores?

- ▶ Listas, pilhas e filas são estruturas **lineares**: guardam os dados em uma única sequência.
- ▶ Diretórios, organogramas e árvores genealógicas têm relações hierárquicas, não lineares.
- ▶ Uma árvore representa essas hierarquias diretamente: um mesmo item pode ter vários outros abaixo dele.



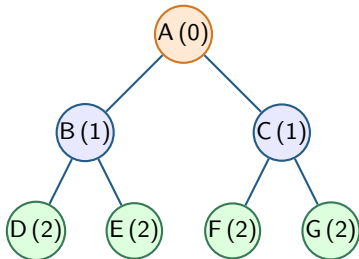
Definição formal de uma árvore

- ▶ Uma árvore é vazia ou consiste em uma raiz e zero ou mais subárvores disjuntas.
- ▶ Cada raiz de subárvore conecta-se à raiz principal por uma aresta.
- ▶ Formalmente: $T = (r, S)$, em que r é a raiz e $S = \{T_1, T_2, \dots, T_n\}$ é o conjunto de subárvores.



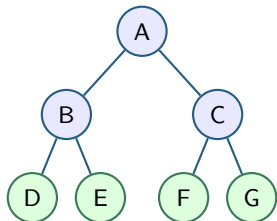
Nó raiz, pai, filho e irmãos

- ▶ **Raiz:** único nó sem pai (A).
- ▶ **Pai e filho:** B é pai de D e E.
- ▶ **Irmãos:** D e E compartilham o mesmo pai, B.
- ▶ **Nível:** distância até a raiz; A está no nível 0.



Folhas, nós internos, ancestral e descendente

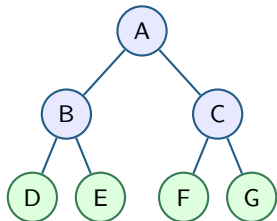
- ▶ **Folha**: não possui filhos (D, E, F, G).
- ▶ **Nó interno**: possui ao menos um filho (A, B, C).
- ▶ u é **ancestral** de v se pertence ao caminho da raiz até v ; v é descendente de u .



A é ancestral de todos os outros nós.

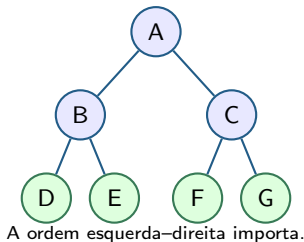
Grau, nível e altura

- ▶ **Grau:** número de filhos; o grau de B é 2.
- ▶ **Nível:** número de arestas desde a raiz.
- ▶ **Altura:** maior nível entre as folhas; aqui, $h = 2$.



Árvore binária

- ▶ Cada nó interno possui **no máximo dois filhos**.
- ▶ Eles são distintos: subárvore esquerda (LST) e direita (RST).
- ▶ Formalmente, $T = (x, L, R)$, em que x é a raiz e L, R são árvores binárias disjuntas.

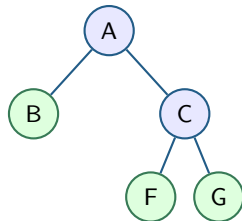


Árvore estritamente binária

Definição

Todo nó possui 0 filhos (folha) ou exatamente 2 filhos; nenhum possui apenas 1 filho.

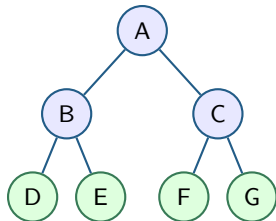
Se há f folhas, há $2f - 1$ nós.



Árvore binária cheia (perfeita)

Definição

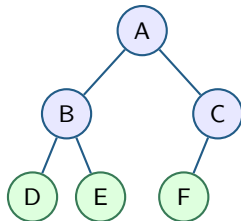
Todo nó interno possui dois filhos e todas as folhas estão no mesmo nível.
Se a altura é h , há $2^{h+1} - 1$ nós.



Árvore binária completa

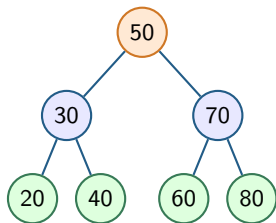
Definição

Todos os níveis, exceto talvez o último, estão preenchidos; no último, os nós ficam o mais à esquerda possível. Essa forma sustenta os heaps binários.



Árvore binária de busca (ABB)

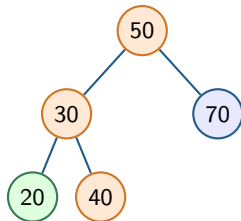
- ▶ Na subárvore esquerda de x , os valores são menores que x .
- ▶ Na direita, os valores são maiores ou iguais a x (convenção adotada aqui).
- ▶ A propriedade de ordem indica qual ramo explorar em cada passo.



Operação de inserção em ABB

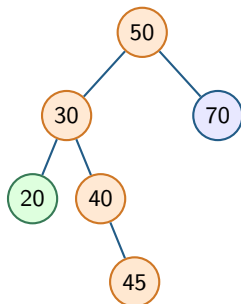
- ▶ A inserção começa na raiz; o novo nó sempre termina como folha.
- ▶ Compare a chave: menor segue à esquerda, maior ou igual segue à direita, até chegar a um ponteiro nulo.
- ▶ Custo médio em árvore balanceada: $O(\log n)$; no pior caso, degenerado: $O(n)$.

Inserção do nó 45: passos 1 e 2



1. $45 < 50$: desça à esquerda.
2. $45 > 30$: desça à direita.

Inserção do nó 45: passos 3 e 4



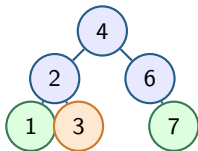
3. $45 > 40$: tente ir à direita.
4. A posição é nula: insira 45.

Operação de remoção em ABB

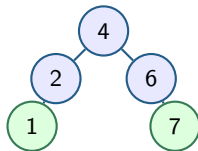
1. **Folha:** desconecte o nó.
2. **Um filho:** conecte o pai diretamente ao filho do nó removido.
3. **Dois filhos:** use o sucessor em ordem (menor da direita) ou o predecessor (maior da esquerda); copie a chave e remova-o do local original.

Remoção: caso 1 (nó folha)

Antes: remover 3

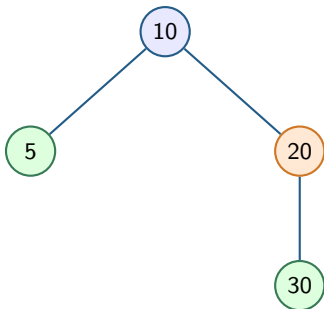


Depois

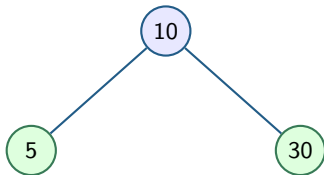


Remoção: caso 2 (um filho)

Antes: remover 20

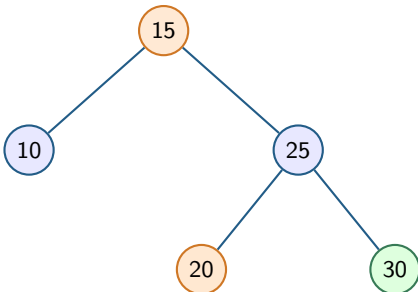


Depois



Remoção: caso 3 (dois filhos)

Ao remover 15, o sucessor em ordem é 20, o menor valor da subárvore direita. Copie essa chave para a raiz e remova a folha 20 original.



Percursos em árvores binárias

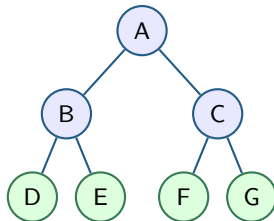
Todos visitam cada nó uma vez: custo $O(n)$. O que muda entre eles é apenas a posição da visita à raiz:

- ▶ Pré-ordem: raiz, esquerda, direita.
- ▶ Em ordem: esquerda, raiz, direita.
- ▶ Pós-ordem: esquerda, direita, raiz.

Percurso em pré-ordem (Raiz, Esquerda, Direita)

1. Visite a raiz.
2. Percorra a esquerda.
3. Percorra a direita.

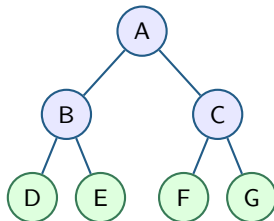
A, B, D, E, C, F, G.



Percurso em ordem (Esquerda, Raiz, Direita)

1. Percorra a esquerda.
2. Visite a raiz.
3. Percorra a direita.

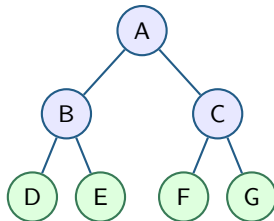
D, B, E, A, F, C, G. Em uma ABB, as chaves saem em ordem crescente.



Percurso em pós-ordem (Esquerda, Direita, Raiz)

1. Percorra a esquerda.
2. Percorra a direita.
3. Visite a raiz.

D, E, B, F, G, C, A.



Implementação de uma ABB em Python

```
class No:
    def __init__(self, chave):
        self.chave, self.esq, self.dir = chave, None, None

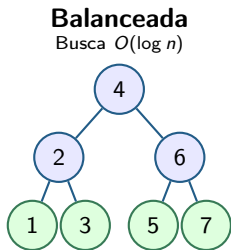
def inserir(raiz, chave):
    if raiz is None: return No(chave)
    if chave < raiz.chave:
        raiz.esq = inserir(raiz.esq, chave)
    else: raiz.dir = inserir(raiz.dir, chave)
    return raiz
```

Percurso em ordem

```
def em_ordem(no):  
    if no is not None:  
        em_ordem(no.esq)  
        print(no.chave, end=" ")  
        em_ordem(no.dir)
```

Em uma ABB, essa função imprime as chaves em ordem não decrescente.

O problema do desbalanceamento



Solução: ABB autobalanceável, como a AVL.

Árvore AVL: definição

- ▶ Uma AVL (Adelson-Velskii e Landis) é uma ABB com uma condição extra sobre a altura das subárvores.
- ▶ $FB(no) = h(\text{direita}) - h(\text{esquerda})$.
- ▶ Em uma AVL, $FB \in \{-1, 0, 1\}$; os valores -2 ou 2 exigem rebalanceamento.
- ▶ Com essa condição, busca, inserção e remoção ficam em $O(\log n)$.

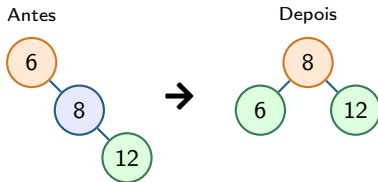
Rebalanceamento com rotações

Uma rotação reorganiza poucos nós, preserva a ordem da ABB e recupera o equilíbrio de altura.

FB do nó	FB do filho pesado	Rotação
+2	0 ou +1	simples à esquerda (caso RR)
+2	-1	dupla direita-esquerda (caso RL)
-2	0 ou -1	simples à direita (caso LL)
-2	+1	dupla esquerda-direita (caso LR)

Rotação simples à esquerda

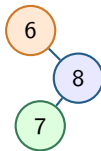
Após inserir 12, o nó 6 fica com $FB(6) = +2$. Como o caminho pesado segue para a direita de 8, temos o caso RR: a rotação coloca 8 na raiz local.



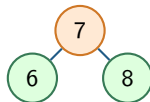
Rotação dupla direita-esquerda

Ao inserir 7, surge um caminho em "joelho": $FB(6) = +2$ e $FB(8) = -1$. Primeiro rotacionamos 8 à direita; em seguida, 6 à esquerda.

Antes

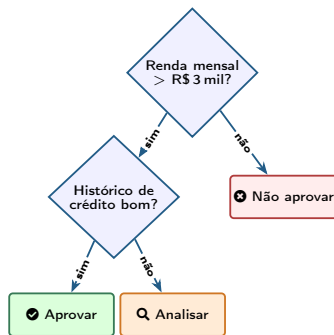


Depois



Árvore de decisão: da estrutura ao modelo

- ▶ No aprendizado supervisionado, cada nó testa uma variável; as arestas registram os resultados possíveis, e as folhas retornam uma classe ou um valor.
- ▶ O treinamento procura divisões que reduzam a impureza (entropia ou índice Gini) ou o erro de regressão.
- ▶ Profundidade limitada, poda e validação em dados separados ajudam a conter o sobreajuste.



Avaliação responsável

- ▶ Separe treino, validação e teste. Ajuste hiperparâmetros somente com os dados de validação.
- ▶ Quando as classes são desbalanceadas, acompanhe precisão, revocação, F1, PR-AUC e a matriz de confusão – acurácia isolada pode esconder erros relevantes.
- ▶ Procure vazamento de alvo e compare o desempenho entre subgrupos que façam sentido para o problema.
- ▶ Decisões de alto impacto exigem revisão humana, registro das escolhas e acompanhamento após a implantação.

Árvores em métodos atuais para dados tabulares

- ▶ Uma árvore isolada é fácil de explicar. Florestas aleatórias combinam muitas árvores para reduzir a variância.
- ▶ Em *gradient boosting*, cada árvore nova corrige os erros do conjunto anterior; XGBoost, LightGBM e CatBoost são implementações amplamente usadas.
- ▶ Esses conjuntos costumam ganhar em desempenho, mas são mais difíceis de explicar do que uma árvore isolada – a comparação deve seguir o protocolo de avaliação do problema.

Fontes: Chen e Guestrin (2016); Ke et al. (2017); Prokhorenkova et al. (2018).

Conclusões

- ▶ Árvores modelam hierarquias; árvores binárias limitam cada nó a dois filhos ordenados.
- ▶ ABBs são eficientes quando a altura é pequena; a AVL garante $O(\log n)$ com rotações.
- ▶ Pré-ordem, em ordem e pós-ordem custam $O(n)$ e servem a objetivos distintos.
- ▶ Árvores de decisão aplicam essa estrutura a classificação e regressão; avaliação e governança são essenciais.

Referências I



Lee, K. D.; Hubbard, S. *Data Structures and Algorithms with Python*. Springer, 2015.



Cormen, T. H. et al. *Introduction to Algorithms*. 3. ed. MIT Press, 2009.



Szwarcfiter, J. L.; Markenzon, L. *Estruturas de Dados e seus Algoritmos*. LTC, 2015.



Breiman, L. Random Forests. *Machine Learning*, 45, 5–32, 2001.



Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. *KDD*, 2016.
<https://doi.org/10.1145/2939672.2939785>



Ke, G. et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *NeurIPS*, 2017. <https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>



Prokhorenkova, L. et al. CatBoost: unbiased boosting with categorical features. *NeurIPS*, 2018. <https://proceedings.neurips.cc/paper/2018/hash/14491b756b3a51daac41c24863285549-Abstract.html>

Estruturas de Dados: Árvores

Professor: Elton Sarmanho¹

✉ eltonss@ufpa.br

¹  Faculdade de Sistemas de Informação – UFPA/CUTINS

7 de setembro de 2026